CAMPUSLINK

Frontend Instruction Guide

PS03 — AI-Powered Placement & Recruitment Management Platform

A page-by-page implementation specification for the public experience, student workspace, recruiter workspace and placement administration.

SOURCE OF TRUTH

Nexaura Prompt Battle — original PS03 (pages 9–11)
CampusLink Build Prompt Pack (21-page technical specification)

Implementation guide • Version 1.0 • 31 October 2026

Document map

Section	What it defines
1. Authority and scope	How the original PS03 and the technical prompt pack govern the frontend.
2. PS03 requirements	The original challenge and basic, advanced and brownie-point requirements.
3. Product roles and information architecture	Role names, route map and navigation.
4. Shared design system	Visual rules, common layout and reusable behavior.
5. Public experience	Scroll-driven university landing page, unified login and registration.
6. Student workspace	Dashboard, profile, resume, jobs, fit, applications, interviews and notifications.
7. Recruiter workspace	Dashboard, job management, applicants, candidate detail, interviews and reports.
8. Placement administration	Overview, recruiter approvals, student records, drives, audit and analytics.
9. Interaction and button inventory	Fixed controls, repeated row actions and button behavior.
10. Frontend architecture and integration	React structure, API behavior, security, states and performance.
11. Responsive and accessibility requirements	Desktop, tablet, mobile, keyboard and reduced motion.
12. Implementation phases	Build order and phase exit criteria.
13. PS03 acceptance checklist	A testable mapping to the original challenge and brownie points.
14. Coding-agent master instruction	Copy-ready standing instruction for frontend implementation.

Important role correction The original PS03 names three roles: students, recruiters and placement administrators. It does not define an Instructor role. The unified login in this guide therefore uses Student, Recruiter and Placement Admin. Do not replace Recruiter with Instructor.

1. Authority, scope and interpretation

1.1 Source hierarchy

Use the supplied original problem statement, “prompt battle.pdf”, PS03 CampusLink on pages 9–11, as the authority for competition scope. Use “CampusLink Build Prompt Pack.pdf” as the detailed technical specification that expands the challenge into models, API contracts, workflows, permissions, AI matching, and development phases. This frontend guide translates those requirements into screen-level instructions; it does not supersede either source.

Source	What it establishes	Frontend consequence
Original PS03, p. 9	Connect students, recruiters and placement administrators; support postings, eligibility, resume matching, applications, shortlisting, interviews and role dashboards.	Every core workflow must be directly reachable and demonstrable in the UI.
Original PS03, p. 10	Basic and advanced functionality, including profile, jobs, eligibility, discovery, application workflow, extraction, semantic matching, skill gaps, notifications, calendar, analytics, exports and audit logs.	The page inventory and component set must expose these features.
Original PS03, p. 11	Brownie points: complete workflow, consistent eligibility, access control, keyword/semantic comparison, upload validation, server-side filtering, explanations and realistic sample data.	The UI must make these criteria visible and testable; backend enforcement remains authoritative.
Build Prompt Pack, §§0–14	Fixed stack, detailed workflows, roles, states, routes, UI patterns, seed data, tests and deployment.	Follow its technical choices and use stored AI/matching results; do not invent a separate backend contract.

1.2 Scope labels used throughout this guide

Label	Meaning	Implementation rule
PS03 REQUIRED	Explicitly listed in the original problem statement.	Must be implemented and demonstrated.
BUILD PACK DETAIL	Specified in the technical prompt pack as an implementation detail.	Implement consistently with the pack unless the project owner explicitly changes it.
USER-DIRECTED EXPERIENCE	A visual/product requirement supplied in the conversation, such as the scroll-driven 3D university landing page.	Include in the frontend experience, but do not misrepresent it as an original PS03 judging requirement.
OPTIONAL EXTENSION	Not supported by the current PS03 or technical API contract.	Do not silently implement. Confirm scope and add backend/API support first.

1.3 Locked frontend stack

The technical prompt pack fixes the client at React 18 + Vite, JavaScript (not TypeScript), React Router 6, TanStack Query, Axios, Tailwind CSS, react-hook-form, Recharts and react-hot-toast. Keep this stack. Do not migrate to Next.js, TypeScript, another router, another data-fetching library or a different CSS system without explicit permission.

The 3D landing experience may require additional packages (for example Three.js / React Three Fiber, Drei and GSAP ScrollTrigger). Treat these as proposed additions that require approval because they are not part of the locked client stack. Do not add them silently.

2. Original PS03 requirements — preserved

PS03 title CampusLink — AI-Powered Placement & Recruitment Management Platform. Difficulty: MEDIUM. Original problem statement: Nexaura Prompt Battle, 31 October.

2.1 Problem context (original wording)

College placement teams manage student profiles, resumes, job descriptions, eligibility requirements and interviews using spreadsheets and separate communication channels. Students may miss suitable opportunities, while recruiters and placement coordinators spend considerable time reviewing applications and organizing recruitment activities.

2.2 The challenge (original wording)

Build CampusLink, a full-stack placement platform connecting students, recruiters and placement administrators. Support job postings, eligibility checks, resume-to-job matching, applications, shortlisting and interview scheduling, with separate role-based dashboards and explainable AI-assisted recommendations.

2.3 Basic functionalities (original wording)

- Role-Based Access: Provide separate interfaces and permissions for students, recruiters and placement administrators.
- Student Profiles: Allow students to maintain their educational details, skills, projects, experience and resume.
- Job Management: Allow recruiters to create, edit, publish and close job openings with descriptions, required skills and eligibility criteria.
- Eligibility Engine: Evaluate job-specific requirements such as branch, CGPA, graduation year and backlogs against student profiles.
- Job Discovery: Allow students to search and filter opportunities by role, location, company and eligibility.
- Application Workflow: Allow eligible students to apply for jobs and track statuses such as Applied, Shortlisted, Interview Scheduled, Selected or Rejected.
- Recruiter Dashboard: Allow recruiters to review applications, filter candidates, shortlist applicants and update application statuses.
- Placement Administration: Allow authorized placement administrators to manage student records and monitor active recruitment drives.

2.4 Advanced functionalities (original wording)

- Provide interview date, time and status information for candidates and recruiters.
- Extract skills and qualifications from uploaded PDF resumes.
- Match equivalent skills using TF-IDF, embeddings or semantic similarity.
- Calculate an explainable resume-to-job matching score.
- Identify skill gaps and recommend relevant learning resources.
- Provide notifications when application statuses change or interviews are scheduled.
- Add calendar views for upcoming interviews and recruitment drives.
- Generate analytics showing in-demand skills, application counts and placement outcomes.
- Export candidate lists and recruitment reports.
- Introduce audit logs for important recruiter and administrator actions.

2.5 Brownie points (original wording)

- Demonstrate the complete workflow: recruiter publishes a job, student applies, recruiter shortlists and the application status updates.

- Enforce eligibility requirements consistently on both the backend and the user interface.
- Prevent unauthorized users from viewing or modifying restricted student and recruitment data.
- Compare keyword-based matching with semantic matching on sample resumes that use different wording for similar skills.
- Validate uploaded resumes and handle missing, malformed or unsupported files.
- Implement pagination, sorting and server-side filtering for candidate lists.
- Provide clear explanations for matching scores and eligibility failures.
- Include realistic sample student profiles, resumes and job postings for a reliable demonstration.

Interpretation The original PS03 does not say that the AI model makes hiring decisions. The UI must present AI-assisted recommendations as explainable support, while eligibility, permissions and the final computed match score come from deterministic backend logic and stored results.

3. Roles and information architecture

3.1 Exact role model

Role label in UI	Purpose	Access model
Student	Maintains a profile and resume, discovers opportunities, checks eligibility and match explanations, applies and tracks recruitment progress.	Own profile and own applications; may browse open jobs.
Recruiter	Creates and manages job openings, reviews applicants, shortlists candidates, updates statuses and schedules interviews.	Own jobs and applicants associated with those jobs. New self-registered recruiters are pending until approved, per the Build Prompt Pack.
Placement Admin	Manages student records and monitors recruitment drives; may administer approvals, analytics and audit information specified in the Build Prompt Pack.	Administrative access enforced by backend role and object-level authorization.

Use the label “Placement Admin” in the UI to make the role clear. “Admin” may be used as the underlying role enum if the existing server uses it. The user’s earlier mention of “instructor” is not supported by original PS03; do not add it as a fourth role.

3.2 Route map

Area	Routes / views	Access
Public	/ (landing), /login, /register	Everyone
Student	/student/dashboard, /student/profile, /student/resume, /student/jobs, /student/jobs/:id, /student/applications, /student/interviews, /student/notifications	Student
Recruiter	/recruiter/dashboard, /recruiter/jobs, /recruiter/jobs/create, /recruiter/jobs/:id/edit, /recruiter/jobs/:id/applicants, /recruiter/interviews, /recruiter/reports	Recruiter
Placement Admin	/admin/dashboard, /admin/recruiters, /admin/students, /admin/drives, /admin/audit-logs, /admin/analytics	Placement Admin
Optional support	/forgot-password, /reset-password	Only if password-reset backend endpoints are implemented

Candidate details should open in a drawer or modal from the recruiter applicant list rather than becoming a separate route unless the application needs a shareable candidate URL. Keep the 24-page/view concept as a planning count, not a requirement to make every panel a route.

3.3 Navigation rules

- Public navigation: CampusLink logo, Explore (scroll anchor), Sign In, Get Started. Keep the main landing CTA obvious.
- Authenticated navigation: persistent role-specific sidebar plus a shared topbar for page title/breadcrumb, notifications and account menu.
- Student sidebar: Overview, My Profile, Resume, Discover Jobs, My Applications, Interviews.

- Recruiter sidebar: Overview, Jobs, Applicants (contextual to a job), Interviews, Reports.
- Placement Admin sidebar: Overview, Recruiter Approvals, Students, Drives, Audit Logs, Analytics.
- Navigation items are presentation only. Every protected page and API must still validate the user role and ownership on the backend.
- On a 401 response, clear the session/token and redirect to login. On 403, show an access-denied state without exposing protected data.

4. Shared design system and application shell

4.1 Visual direction

Use a premium, welcoming university and enterprise visual language. The landing page may be cinematic and image-led, but authenticated dashboards should be clean, compact and task-oriented. Avoid a neon-heavy science-fiction look, excessive gradients, decorative motion in tables, and game-like controls.

Token	Recommended value	Usage
Primary navy	#142B45	Sidebar, titles, primary actions and trust-oriented brand elements.
Teal	#0F766E	Active states, links, progress accents and positive workflow cues.
White	#FFFFFF	Main content surfaces and forms.
Soft background	#F1F5F9	Application background and secondary panels.
Text	#243447	Body copy and table content.
Muted text	#5D6B78	Helper text, timestamps and secondary metadata.
Warning / danger	Amber / muted red	Pending states, validation and destructive actions.

Typography: use Inter or Manrope if already approved and installed; otherwise use the project's existing sans-serif stack. Use a clear type scale and consistent heading hierarchy. Cards should use restrained rounding (roughly 10–14 px), subtle borders and adequate whitespace.

4.2 Shared application shell

- Left sidebar: role logo/brand, role navigation, active navigation indicator and optional collapsed mode on tablet.
- Topbar: current page title or breadcrumb, notification icon with unread count, profile avatar/name and account menu.
- Main content: page title, short context line, primary action in a consistent location, then content sections.
- Use responsive tables that become cards or horizontally scrollable tables on narrow screens; do not simply shrink desktop tables until unreadable.
- Persist filters in query parameters where appropriate so refresh and browser navigation do not unexpectedly clear a user's context.

4.3 Shared reusable components

Component	Responsibility	Required behavior
AppShell / Sidebar / Topbar	Consistent role layout and navigation.	Role-aware menu; active route; responsive collapse.
StatCard	Key dashboard counts.	Show value, label, optional trend only when supported by data.
JobCard	Job discovery item.	Company, role, location, type, deadline, eligibility and stored match score.
StatusBadge	Application/job/interview status.	Use consistent labels and color; always include text, not color alone.

Component	Responsibility	Required behavior
EligibilityPanel	Eligibility result and failures.	Display exact backend reasons; no client-only eligibility calculation.
MatchScore / ScoreBreakdown	Matching score and evidence.	Show method, components, matched and missing skills, evidence.
MethodComparison	Keyword, TF-IDF and semantic/embedding comparison.	Same job and same resume; labels explain each method.
DataTable / Pagination / Filters	Applicants, jobs, students, audit logs.	Server-side filtering, sorting and pagination.
Modal / ConfirmDialog / Drawer	Secondary detail and irreversible actions.	Accessible focus management; explicit cancel/confirm.
ResumeViewer	PDF preview/download.	Loading, unsupported file and preview failure states.
Calendar / InterviewCard	Upcoming interviews and drive dates.	Show date, time, status and local timezone.
Skeleton / EmptyState / ErrorState	Asynchronous and no-data feedback.	Every data-driven page must include all relevant states.
Toast	Mutation feedback.	Success/error toasts for create, edit, delete, upload and status changes.

5. Public experience and authentication

5.1 Landing page — user-directed signature experience

Design objective The public landing page should introduce CampusLink as a connected university placement ecosystem. The requested signature interaction is a scroll-driven journey: as the visitor scrolls, the camera moves progressively deeper into a university model and reveals the placement workflow. This is a user-directed design requirement, not a separate original PS03 functional criterion.

Page structure

Scroll segment	Scene / camera direction	On-screen message and purpose
0–20%	Elevated aerial view of the complete university campus. Establish the campus as the visual anchor.	Hero: “Your campus. Connected to opportunity.” Brief CampusLink description. CTA: Explore Campus / Get Started.
20–35%	Camera approaches the main gate and central entrance.	Introduce the community: students, recruiters and placement administrators.
35–50%	Camera travels past academic buildings and student areas.	Student profiles, skills, projects, experience and resumes.
50–65%	Camera moves into a lab/project space.	Resume skill extraction and explainable job matching; avoid claiming AI guarantees placement.
65–80%	Camera moves toward the placement / recruitment centre.	Job postings, eligibility checks, applications, shortlisting and interview scheduling.
80–100%	Camera reaches a central interior / campus focal point and settles.	Final CTA: Sign In / Get Started. Footer links and contact/help information.

Motion and interaction rules

- Pin the visual canvas during the story sequence while the document itself continues to scroll. Use scroll progress to interpolate camera position, camera target, selected highlights and text transitions.
- Treat all scenes as one continuous journey through one campus model. Do not use six unrelated full-screen background swaps as a substitute for camera movement.
- Use short text overlays that enter and leave at deliberate points. Keep copy readable and avoid placing essential text over visually busy model areas.
- Keep primary navigation accessible throughout the sequence. The visitor must be able to reach Sign In without completing the animation.
- Add a small “Skip to content” or “Skip animation” action. Respect prefers-reduced-motion and provide a static/non-3D fallback.
- Lazy-load the 3D assets only on the landing page. If WebGL fails, show a high-quality static campus hero with the same content and CTAs.
- Provide a mobile alternative: simplified camera path or static image sequence; do not force a large desktop 3D scene onto low-powered devices.

Landing page action inventory

Control	Count	Behavior
Explore Campus	1	Scrolls to or starts the campus story; if already at the hero, begins the scroll journey.
Sign In	1	Navigates to /login.
Get Started	1	Navigates to /register.
Explore Opportunities	1	Scrolls to the placement workflow / opportunity section; it does not promise live job data to unauthenticated visitors.
Skip Animation / Skip to Content	1	Moves to the next content section and disables the cinematic sequence for that visit.

Optional navigation anchors (About, How it Works, Contact) should be links, not oversized CTA buttons. Do not add many competing hero actions.

5.2 Unified login page

Create one login route for all three PS03 roles. Use a role selector with Student, Recruiter and Placement Admin. The role selector is an account-type hint for the user experience; the authenticated role returned by the backend is authoritative.

Element	Count	Specification
Role selector	3 options	Student / Recruiter / Placement Admin. A single segmented control or accessible radio group.
Email field	1	Required; validate email format.
Password field	1	Required; show/hide control.
Show / hide password	1	Toggle visibility without clearing the password.
Remember me	1 checkbox	Only if session persistence policy supports it.
Forgot password	1 link	Optional extension; only active when reset API exists.
Sign In	1 primary button	Submit once; show pending state; handle invalid credentials and pending/suspended accounts.
Create account	1 link	Navigate to registration.

On successful login, obtain the real user role from the server and redirect to that role's dashboard. If the selected role differs from the server role, show a clear mismatch message or simply use the server role consistently. Never trust a client role selection to grant access.

5.3 Registration page

Provide one registration route with an explicit Student / Recruiter account type selector. Admin/Placement Admin registration must not be public; the Build Prompt Pack states that administrator accounts are created only by the seed script.

Registration path	Fields	Primary actions
Student	Full name, email, password, confirm password, roll/registration number (if collected at signup), terms acknowledgement if required.	Create Student Account; Back to Login.
Recruiter	Full name, email, password, confirm password, company, designation; website optional.	Create Recruiter Account; Back to Login.

After recruiter registration, display a prominent pending-approval message. A pending recruiter may sign in only if the backend permits it, but must not be shown publish controls that imply they can publish before approval. Backend authorization remains mandatory.

5.4 Password reset — optional extension

The original PS03 and the current Build Prompt Pack do not specify reset endpoints or email-verification flows. Do not present password recovery as an implemented feature unless the backend contract exists. If approved, add request-reset and reset-password views with clear expired-link and success states.

6. Student workspace

The student workspace must make three questions easy to answer: What is my placement profile status? Which opportunities can I apply for? What is happening with my applications? The job discovery and fit pages are especially important because they expose eligibility and explainable matching.

6.1 Student dashboard

Purpose: Give a personalized overview of placement readiness and current recruitment activity.

Content and layout

- Welcome header with student name, branch and graduation year when available.
- Profile completeness indicator based on required profile fields; do not equate completeness with job eligibility.
- Recent applications with company, job, applied date and current status.
- Upcoming interviews with date, time, mode and status.
- Recommended/open jobs using backend-provided stored ranking data; show eligibility separately from match score.
- Helpful next step when profile or resume is incomplete.

Actions and controls

Control / action	Count guidance	Expected behavior
Complete Profile	1	Navigates to My Profile.
Upload / Manage Resume	1	Navigates to Resume.
Explore Jobs	1	Navigates to Discover Jobs.
View Applications	1	Navigates to My Applications.

Required states

- Skeletons for summary cards and lists.
- Empty state when no applications, interviews or recommended jobs exist.
- Backend/API error state with retry.
- Do not display invented placement counts or fake match values.

6.2 Student profile

Purpose: Allow students to maintain the profile information used for job eligibility and matching.

Content and layout

- Personal details: name, email (read-only if account-managed), phone and roll number.
- Academic details: branch, CGPA, graduation year and backlog count.
- Skills as editable chips with add/remove interaction.
- Projects: title, description and technology stack; allow multiple entries.
- Experience: company, role, description, start and end dates; allow multiple entries.
- Clearly distinguish saved data from unsaved edits.

Actions and controls

Control / action	Count guidance	Expected behavior
Save Profile	1	Validates and saves all changed profile fields.
Add Project	1	Adds a project form section; each entry has a remove action.
Add Experience	1	Adds an experience form section; each entry has a remove action.
Remove entry	Repeated	Remove only the selected project/experience; confirm if unsaved data would be lost.

Required states

- Field-level validation and server validation errors.
- Saving/loading state and success toast.
- Empty projects/experience state that invites adding without implying they are mandatory.
- Profile completeness recalculates from persisted values.

6.3 Resume management

Purpose: Let students upload, inspect and reprocess a PDF resume, and see extracted information.

Content and layout

- Drag-and-drop and file-picker upload zone.
- File name, size, upload date and available download/preview action.
- Parsing state: none, processing, ok or failed, mapped to clear human language.
- Extracted skills, projects, experience and qualifications when returned by backend.
- Explain PDF-only and 5 MB maximum from the Build Prompt Pack; show validation errors clearly.
- Retry/reparse affordance for failures.

Actions and controls

Control / action	Count guidance	Expected behavior
Upload / Replace Resume	1	Selects PDF and uploads it; disables during upload.
Re-parse Resume	1	Calls the reparse endpoint; displays processing state.
Download Resume	1	Downloads the stored PDF where authorized.
Preview Resume	1 optional	Opens the in-app PDF viewer if browser/file support allows it.

Required states

- Missing file, wrong type, malformed PDF and oversize file errors.
- Uploading / processing state.
- Parse success with extracted skills.
- Parse failure with parseError summarized safely and a retry option.

- Never show parsed resume text as trusted instructions or expose it to another student.

6.4 Job discovery

Purpose: Allow students to search and filter available opportunities.

Content and layout

- Search by job title/role or keyword.
- Filters for location, company, job type and eligibility; include branch/CGPA context only when exposed by API.
- Tabs or segmented selector for All Jobs and Recommended for You.
- Job cards show title, company, location, type, deadline, required skills, eligibility status and stored match score where available.
- Server-side pagination and sorting; preserve filters while navigating pages.

Actions and controls

Control / action	Count guidance	Expected behavior
Search	1	Submits or debounces search; URL/query state should represent current filters.
All Jobs / Recommended	2 options	Switches dataset/view without changing user role.
Filter controls	Variable	Apply supported filter values server-side.
Clear Filters	1	Resets active filters.
View Details	1 per card	Opens job detail and fit page.

Required states

- Loading skeleton cards.
- No results state with clear-filter suggestion.
- Error state and retry.
- Do not compute embeddings or invoke LLM on the client or per list request; show server-stored scores only.

6.5 Job details and AI fit

Purpose: Explain the opportunity, eligibility and resume-to-job fit before the student applies.

Content and layout

- Job details: company, title, description, location, type, salary if supplied, skills, deadline and drive date.
- Eligibility panel: eligible/ineligible plus each failure reason, such as branch, CGPA, graduation year or backlogs.
- Match score and method label.
- Score breakdown: skill coverage, document similarity and project/experience relevance where the API provides them.
- Matched skills and missing skills with evidence; show the resume phrase or item used as evidence.
- Skill-gap learning resources (one or two resources per gap where configured).
- Method comparison: keyword, TF-IDF and embeddings for the same job/resume pair where the compare endpoint is available.

Actions and controls

Control / action	Count guidance	Expected behavior
Apply Now	1	Enabled only when backend eligibility says eligible and job is open; POST apply; show duplicate/422 errors.
Back to Jobs	1	Returns to discovery and preserves filters.
Compare Methods	1 optional	Loads keyword/TF-IDF/embedding comparison for same job and student.
Learning resource link	Repeated	Opens the resource URL supplied by the application.

Required states

- Eligibility loading and unavailable profile data state.
- Ineligible state with exact failures and disabled apply.
- Eligible state with active apply action.
- Already applied state; never offer duplicate application as a new action.
- Matching unavailable/fallback state with method clearly labelled.
- Do not imply a match score is a guarantee of selection.

6.6 My applications

Purpose: Let students see the status and history of each application.

Content and layout

- Paginated list of applications with company, job, applied date, score and current status.
- Application timeline using statusHistory.
- Optional detail drawer with eligibility snapshot, match explanation and recruiter status notes if returned by API.
- Use the exact state labels: Applied, Shortlisted, Interview Scheduled, Selected, Rejected.

Actions and controls

Control / action	Count guidance	Expected behavior
View Details	1 per row	Opens application details/timeline.
Pagination	Variable	Changes page using server pagination.

Required states

- No applications state.
- Loading and API error states.
- Status change reflected after refresh/refetch; do not optimistically invent a transition.
- Selected and Rejected are terminal states in the specified state machine.

6.7 Interviews and calendar

Purpose: Show scheduled interview information for students.

Content and layout

- List and calendar views for upcoming and past interviews.
- Date, time, duration, online/onsite mode, meeting link or location and current status.
- Time displayed in the browser's local timezone; timestamps are stored in UTC per the Build Prompt Pack.
- Filter or navigate by month/week when calendar endpoint supports it.

Actions and controls

Control / action	Count guidance	Expected behavior
List / Calendar	2 view options	Switch between agenda list and calendar.
View Interview	1 per item	Opens full details.
Open Meeting Link	Conditional	Opens supplied online interview link.
Reschedule / Cancel	Only if allowed	Show only when the workflow/API grants student action; do not assume students can change recruiter appointments.

Required states

- No upcoming interviews state.
- Loading/error states.
- Cancelled, completed and rescheduled status states.
- Timezone label or unambiguous local display.

6.8 Notifications

Purpose: Notify students about application changes and scheduled interviews.

Content and layout

- Chronological notification list with title, message, date and read/unread state.
- Links to relevant job, application or interview.
- Unread count in the topbar.

Actions and controls

Control / action	Count guidance	Expected behavior
Mark All as Read	1	Marks all current user notifications read.
Mark as Read	1 per unread item	Updates a single notification.
Open Notification	1 per item	Navigates to related resource when a link exists.

Required states

- Empty notification state.
- Unread and read states.
- Mutation success/error feedback.
- Only the authenticated user's notifications may be displayed.

7. Recruiter workspace

The recruiter experience must support the complete employer workflow: create a job, configure skills and eligibility, publish it, review server-filtered applicants, inspect explainable match evidence, shortlist and schedule interviews. A pending recruiter must not be able to publish jobs.

7.1 Recruiter dashboard

Purpose: Summarize the recruiter's own open jobs and applicant pipeline.

Content and layout

- Open jobs, total applicants, shortlisted candidates and upcoming interviews.
- Job-wise pipeline counts and recent applicant activity.
- Quick links to job creation, jobs and applicants.
- If recruiter status is pending, show approval status and disable publish workflow with explanation.

Actions and controls

Control / action	Count guidance	Expected behavior
Create Job	1	Opens job creation.
View Jobs	1	Opens recruiter's own job list.
View Applicants	1	Opens applicants for a selected owned job.

Required states

- Pending approval state.
- No jobs / no applicants states.
- Loading/error states.
- Metrics are scoped to the signed-in recruiter only.

7.2 Job management

Purpose: Allow an approved recruiter to manage their own job openings.

Content and layout

- Table/list with job title, company, location, type, application deadline, status and applicant count.
- Statuses: Draft, Open, Closed.
- Search/filter/sort and pagination where available.
- Contextual actions vary by job status.

Actions and controls

Control / action	Count guidance	Expected behavior
Create Job	1 global	Opens create form.
Edit	Per owned editable row	Opens edit form.
Publish	Per draft row	Publishes after validation; only approved recruiter.
Close	Per open row	Closes job after confirmation.
Delete Draft	Per draft row	Deletes only draft jobs after confirmation.

Control / action	Count guidance	Expected behavior
View Applicants	Per job row	Opens applicants for that job.

Required states

- No jobs state.
- Pending recruiter approval state.
- Draft/open/closed states.
- 403 on cross-owner access; do not show another recruiter's records.

7.3 Create / edit job

Purpose: Collect all information required to publish a job and compute eligibility/matching.

Content and layout

- Basic details: title, description, company, location, full-time/internship type and salary range if used.
- Required skills as editable chips.
- Eligibility builder: allowed branches, minimum CGPA, graduation year and maximum backlogs.
- Recruitment dates: application deadline and drive date.
- AI skill suggestion results displayed as editable suggestions; recruiter reviews and edits before publishing.
- Clear distinction between Save Draft and Publish.

Actions and controls

Control / action	Count guidance	Expected behavior
Suggest Skills	1	Sends job description to suggestion endpoint; fills reviewable skill chips, never auto-publishes.
Add Skill	1 control	Adds a manually entered skill.
Remove Skill	Per chip	Removes selected skill.
Save Draft	1	Persists draft.
Publish Job	1	Validates and publishes; permitted only for approved recruiter.
Cancel	1	Returns to job list; warn about unsaved changes.

Required states

- Field validation and server errors.
- AI suggestion loading, success and fallback states.
- Pending recruiter approval state.
- Publishing confirmation and success toast.
- Do not permit invalid eligibility ranges or past deadlines if backend rejects them.

7.4 Applicants list

Purpose: Allow recruiters to review and filter applicants for one of their own jobs.

Content and layout

- Server-side paginated table with candidate name, branch, CGPA, match score, application status and applied date.
- Sort by score or date; filter by status and minimum score; search by supported query.
- Candidate row opens a detail drawer.
- Bulk selection and bulk shortlist action.
- CSV export for the current job when permitted.

Actions and controls

Control / action	Count guidance	Expected behavior
Search / Filters	Variable	Submit supported filters to server; preserve them in route/query state.
Sort	1 selector	Sort by match score or application date.
Shortlist Selected	1 global	Updates selected valid applicants; show per-item results if partial failure occurs.
Export CSV	1 global	Downloads current job's authorized candidate list.
View Candidate	1 per row	Opens candidate drawer.

Required states

- Empty state when there are no applicants.
- Loading state and API error.
- Pagination, sorting and filters operate server-side.
- Do not call LLM or embedding model while listing candidates; use stored score and summaries.
- Cross-owner requests return 403 and no candidate data is rendered.

7.5 Candidate detail drawer — component, not mandatory route

Open from an applicant row. Keep it focused and scrollable; do not turn it into a second dashboard.

- Candidate identity and academic details permitted for this recruiter because the student applied to this recruiter's job.
- Skills, projects, experience and permitted resume preview/download.
- Match score, scoring method and component breakdown.
- Matched/missing skills and per-skill evidence.
- AI-assisted 2–3 sentence summary based only on computed facts. Label fallback/template output when applicable.
- Application status history and eligibility snapshot.

Action	Count	Behavior
Shortlist	1	Moves Applied to Shortlisted if valid.
Reject	1	Moves an allowed non-terminal state to Rejected after confirmation.
Schedule Interview	1	Opens scheduler for this application.
Close Drawer	1	Returns to applicant table with filter/page state retained.

State-machine restriction Allowed transitions from the Build Prompt Pack: Applied → Shortlisted or Rejected; Shortlisted → Interview Scheduled or Rejected; Interview Scheduled → Selected, Rejected or back to Shortlisted if an interview is cancelled. Selected and Rejected are terminal. Invalid transitions should show the backend's 409 response.

7.6 Recruiter interviews

Purpose: Schedule and manage interviews associated with applications.

Content and layout

- Interview list/calendar with candidate, job, date/time, duration, mode, location/link and status.
- Schedule form requires application/candidate, date/time, duration and online/onsite mode.
- For online interviews collect meeting link; for onsite collect location.
- Check recruiter double-booking conflict returned by backend.
- Reschedule, cancel and complete according to current status.

Actions and controls

Control / action	Count guidance	Expected behavior
Schedule Interview	1	Creates interview for a shortlisted applicant.
Reschedule	Per eligible interview	Updates date/time/details.
Cancel	Per scheduled interview	Confirms cancellation and triggers supported status behavior.
Mark Completed	Per scheduled interview	Marks interview complete.
Open Calendar	1 view	Shows interviews and dates.

Required states

- Conflict error for recruiter double booking.
- Required-field validation.
- Success/error toast.
- Application moves to Interview Scheduled when scheduling succeeds, as specified.

7.7 Recruitment reports

Purpose: Show the recruiter's own job performance and recruitment funnel.

Content and layout

- Applicant count; counts per application status; average and median match score.
- Top applicant skills and most common missing skills.
- Interview counts, selection rate and applications per day.
- Funnel chart and score distribution chart.
- All report data scoped to recruiter-owned jobs.

Actions and controls

Control / action	Count guidance	Expected behavior
Select Job	1 selector	Loads the selected owned job report.
Export CSV	1	Downloads report for the selected job.
Date/filter controls	Optional	Only include if supported by report endpoint.

Required states

- No report data state.
- Loading/error states.
- Charts must reflect API data; do not hardcode graph series.
- Unauthorized job IDs must not reveal report data.

8. Placement administration workspace

The original PS03 explicitly calls this role “placement administrators”. The technical Build Prompt Pack sometimes shortens the server role to “admin”. Label the UI “Placement Admin” while mapping to the server’s authorized admin role.

8.1 Placement Admin overview

Purpose: Give placement administrators a system-wide view of recruitment activity.

Content and layout

- Total students, recruiters, pending recruiter approvals and active drives.
- Applications and placement outcomes, grouped by job/branch where available.
- In-demand skills and recruitment activity summaries.
- Quick links to approvals, student records and drives.

Actions and controls

Control / action	Count guidance	Expected behavior
Review Recruiter Approvals	1	Opens pending recruiter list.
Manage Students	1	Opens student administration.
View Drives	1	Opens active/upcoming/closed drives.

Required states

- Loading and API error states.
- Empty/new-system state.
- Every metric must be backed by real API/seed data.

8.2 Recruiter approvals

Purpose: Allow authorized placement administrators to approve recruiters.

Content and layout

- Pending recruiter list with name, email, company, designation, website and registration date.
- Approval status and review details.
- Clear explanation that approved recruiters can publish jobs.

Actions and controls

Control / action	Count guidance	Expected behavior
Approve	1 per pending recruiter	Approves recruiter; refreshes status and reports success.
Reject	1 per pending recruiter	Rejects only if backend supports this state/action; otherwise do not show.
View Details	1 per row	Shows recruiter details.

Required states

- No pending approvals state.

- Approve loading and success state.
- Failure state; preserve current row.
- Only Placement Admin can call the approval endpoint.

The Build Prompt Pack explicitly specifies recruiter approval; rejection is not clearly specified as an endpoint/state. Show Reject only if it is implemented and authorized.

8.3 Student management

Purpose: Allow placement administrators to search and manage student records.

Content and layout

- Paginated search/filter by branch, CGPA and graduation year as supported.
- Student name, roll number, branch, CGPA, graduation year and account status.
- View permitted student details and update/suspend where the backend supports it.
- CSV export when specified by the admin API.

Actions and controls

Control / action	Count guidance	Expected behavior
Search / Filter	Variable	Uses server-side query parameters.
View Student	1 per row	Opens authorized details.
Edit	Conditional	Updates only permitted fields.
Suspend / Reactivate	Conditional	Requires confirmation and backend support.
Export Students CSV	1	Downloads authorized export.

Required states

- Empty/loading/error states.
- 403 or 404 states do not leak record data.
- Pagination and filters are server-side.
- Do not expose password hashes or unnecessary sensitive information.

8.4 Recruitment drives

Purpose: Monitor active and historical recruitment drives.

Content and layout

- List of jobs/drives with company, title, status, deadline, drive date and applicant count.
- Filters for active, upcoming and closed drives.
- Detail view showing pipeline and drive activity.

Actions and controls

Control / action	Count guidance	Expected behavior
View Drive	1 per row	Opens drive details.
Manage Drive	Conditional	Only if the admin API supports the required action.

Required states

- No drives state.
- Loading/error states.
- Drive status derived from backend data, not guessed from client clock alone.

8.5 Audit logs

Purpose: Let administrators inspect important recruiter and administrator actions.

Content and layout

- Actor, actor role, action, target type/ID, timestamp and metadata/IP where permitted.
- Filters by actor, action and date range.
- Detail view for metadata; avoid displaying secrets or unnecessary personal data.

Actions and controls

Control / action	Count guidance	Expected behavior
Apply Filters	1	Applies filters server-side.
Clear Filters	1	Resets filter state.
View Log Details	1 per row	Opens metadata drawer/modal.
Pagination	Variable	Uses server pagination.

Required states

- No logs state.
- Loading/error states.
- Restricted access state.
- Do not allow editing/deleting audit records unless a separate requirement exists.

8.6 Admin analytics

Purpose: Provide system-wide analytics for placement administrators.

Content and layout

- Applications per job.
- In-demand skills based on job requirements and applicant skills.
- Placement outcomes by branch.
- Application funnel counts and placement outcomes.
- CSV exports where the API provides them.

Actions and controls

Control / action	Count guidance	Expected behavior
Date / scope filters	Optional	Only filters supported by backend.
Export CSV	1 per exportable report	Downloads server-generated report.

Required states

- Charts render backend values.

- Loading, empty and error states.
- Avoid fake percentages or totals.
- Use accessible chart labels and text summaries.

9. Interaction and button inventory

9.1 Fixed action inventory by view

Counts below are the planned fixed/global controls for each view. Repeated table rows, job cards, skill chips and dynamic project/experience entries create additional contextual controls. Do not inflate a page with buttons merely to match a numeric total.

View	Primary/fixed actions	Repeated/contextual controls
Landing	Explore Campus; Sign In; Get Started; Explore Opportunities; Skip Animation	Navigation anchor links.
Login	Role selector (3 options); Sign In; show/hide password; Remember me; Create Account	Forgot password link only if supported.
Register	Account type selector (Student/Recruiter); Create Account; Back to Login	Show/hide password; optional terms checkbox.
Student dashboard	Complete Profile; Upload/Manage Resume; Explore Jobs; View Applications	Links on recent application/interview items.
Student profile	Save Profile; Add Project; Add Experience	Remove each project/experience; add/remove skill.
Student resume	Upload/Replace; Re-parse; Download; optional Preview	File picker / drag-drop.
Job discovery	Search; All Jobs/Recommended; Clear Filters	Filters, pagination, View Details per card.
Job fit/detail	Apply Now; Back to Jobs; optional Compare Methods	Learning resource links.
My applications	View Details per row	Pagination.
Student interviews	List/Calendar toggle	View interview; meeting link; conditional actions.
Notifications	Mark All as Read	Mark as Read and Open Notification per item.
Recruiter dashboard	Create Job; View Jobs; View Applicants	Job/activity links.
Job management	Create Job	Edit, Publish, Close, Delete Draft, View Applicants per row.
Create/edit job	Suggest Skills; Add Skill; Save Draft; Publish; Cancel	Remove skill chips.
Applicants	Shortlist Selected; Export CSV	Search, filters, sort, pagination, View Candidate per row.
Candidate drawer	Shortlist; Reject; Schedule Interview; Close Drawer	Resume open/download and detail links.
Recruiter interviews	Schedule Interview; Open Calendar	Reschedule, Cancel, Mark Completed per eligible interview.
Recruiter reports	Select Job; Export CSV	Optional supported report filters.
Admin overview	Review Approvals; Manage Students; View Drives	Dashboard links.
Recruiter approvals	Approve	View Details; Reject only if backend supports it.
Student management	Search/Filter; Export Students CSV	View, Edit, Suspend/Reactivate if supported.
Drives	Filter tabs	View Drive; Manage Drive only if supported.

View	Primary/fixed actions	Repeated/contextual controls
Audit logs	Apply Filters; Clear Filters	View details; pagination.
Admin analytics	Optional supported filters; Export CSV	Per-report export actions.

9.2 Universal button behavior

- One visually dominant primary action per content section. Secondary actions should not compete with it.
- Every button must have a real handler and a defined result. Do not render inert controls or “coming soon” buttons in the completed workflow.
- Disable submit actions during requests; prevent double submission.
- Use confirmation dialogs for destructive operations (delete draft, reject, suspend, cancel interview) and clearly name the affected record.
- Show success/error toast for mutations and field-level errors for forms.
- Use text plus icon for important actions. Icon-only controls need accessible names and tooltips.
- When backend rejects a transition with 409 or an apply with 422, display the server-provided explanation in human-readable form.

10. Frontend architecture and backend integration

10.1 Suggested client folder organization

Use the existing client folder layout from the Build Prompt Pack. The tree below is a practical interpretation; do not create a second app or duplicate the server workspace.

```
client/
  src/
    main.jsx
    app/
      router.jsx
      providers.jsx
      queryClient.js
    api/
      axios.js
      auth.api.js
      students.api.js
      jobs.api.js
      applications.api.js
      interviews.api.js
      notifications.api.js
      analytics.api.js
      admin.api.js
    components/
      layout/  forms/  feedback/  tables/  charts/
    features/
      auth/ student/ recruiter/ admin/ jobs/ applications/ interviews/ reports/
    layouts/
      PublicLayout.jsx
      StudentLayout.jsx
      RecruiterLayout.jsx
      AdminLayout.jsx
    hooks/
    utils/
    styles/
```

10.2 Data and API behavior

- Use Axios for HTTP requests and one configured base URL from the Vite environment. Do not hardcode the deployed API URL.
- Use TanStack Query for server state, caching, invalidation, refetching and loading/error state. Keep transient form/UI state local.
- Use React Hook Form for forms. Validate required inputs client-side for usability, but treat Zod/server validation as authoritative.
- Follow the backend response convention: success { success: true, data }; failure { success: false, error: { code, message, details? } }.
- List views must use server pagination, sorting and filtering and display returned metadata: page, limit, total and totalPages.
- On mutation success, invalidate/refetch affected queries so status badges, counts and list rows reflect persisted backend data.
- Never compute the final eligibility or final match score only in the browser. Display the backend result and its explanations.
- Never call LLM or embedding services from the browser. Never request a new embedding per list row. Display stored match results returned by the API.
- Use the same job and student pair when presenting keyword vs TF-IDF vs embeddings comparison.

- Convert UTC timestamps returned by the backend to local display time, preserving correct date/time semantics.

10.3 Security and access control

- Keep the token handling mechanism consistent with the backend contract; attach the Bearer token through the Axios interceptor if the server uses the specified JWT header.
- Route guards improve navigation but are not security boundaries. Backend role checks and object-level ownership checks are mandatory.
- Never expose passwordHash, embedding vectors or private resume data in normal list views.
- Students must see only their own profile and applications. Recruiters must see only jobs they own and students who applied to those jobs. Placement Admin access follows the backend permissions.
- On 401 clear authentication state and redirect to login. On 403 show a permission message. On 404 show not found. Do not leak IDs or private record details in error UI.
- Treat resume content as untrusted data. The frontend must not execute, render as HTML or follow instructions embedded in parsed resume text.

10.4 Error, loading and empty states

State	Required UI pattern	Example
Loading	Skeletons for expected content; spinner only for small actions.	Applicants table loading; job cards skeleton.
Empty	Explain why there is no data and provide a relevant next action.	No jobs match filters → Clear Filters.
Validation	Inline message next to field plus summary for long forms.	CGPA must be between 0 and 10.
Unauthorized	Access denied; no protected data.	Recruiter tries another recruiter's applicant URL → 403 screen.
Not found	Clear record-not-found page.	Job ID no longer exists.
Conflict	Explain state/duplicate conflict.	Duplicate application or invalid status transition → 409.
Eligibility failure	Show exact failures from backend.	Apply rejected with 422 and CGPA/branch/backlog reasons.
Upload failure	Explain invalid type, size or malformed file; preserve retry.	Unsupported file / over 5 MB / corrupt PDF.
Server failure	Friendly retry state; never show stack traces.	Temporary API failure.

11. Responsive, accessibility and performance requirements

11.1 Responsive behavior

Viewport	Required behavior
Desktop	Full role sidebar, topbar and content grid. Tables can show all defined columns. Landing page uses full cinematic experience if hardware supports it.
Tablet	Sidebar collapses to a drawer or compact rail. Dashboard cards reflow. Forms stack. Tables may scroll horizontally with sticky key columns where practical.
Mobile	Sidebar becomes a menu drawer; primary actions remain reachable; forms use single-column layout; tables become stacked cards or a clearly scrollable region; dialogs fit viewport; landing scene uses simplified/static rendering.

11.2 Accessibility

- Use semantic headings, buttons, labels, links and table headers.
- All form controls need visible labels; placeholders are not labels.
- All controls must be keyboard accessible with visible focus indicators.
- Modals and drawers must manage focus, support Escape where appropriate and return focus to the invoking control.
- Status must not be communicated by color alone; show visible text labels.
- Charts need titles, legends and a text/table summary for key values.
- Respect prefers-reduced-motion; provide a non-animated alternative for scroll-driven scenes.
- Ensure sufficient text/background contrast and reasonable tap target sizes.

11.3 Performance

- Lazy-load the 3D model and route-level bundles. Keep Three.js out of authenticated dashboard bundles if not required there.
- Optimize GLB/glTF geometry, texture dimensions and compression. Avoid loading the model before the landing hero is needed.
- Use image dimensions and responsive sources to avoid layout shifts.
- Paginate lists on the server; avoid downloading all candidates/students/jobs to filter in the browser.
- Cache API data with TanStack Query and invalidate narrowly after mutations.
- Do not run PDF parsing, embeddings or matching in the browser; these are backend/worker responsibilities.
- Provide a simple fallback for reduced motion, no WebGL and slow/low-memory devices.

12. Frontend implementation phases

Implement in thin, verifiable slices. Do not build every static page first and leave API integration until the end. Each phase must include a browser test and the exit criteria below.

Phase	Scope	Exit criteria
F1 — Foundation	Vite/React app shell, Tailwind tokens, router, Axios client, TanStack Query, auth context, shared components, responsive role layouts.	App boots; routes render; role-protected layouts behave; common feedback components work.
F2 — Public & auth	Landing page story and fallback; unified login; student/recruiter registration; server-authenticated redirects.	Visitor can explore, register/login and land in the correct role dashboard; admin signup is not public.
F3 — Student	Dashboard, profile, resume upload/status/reparse, job discovery, job detail/fit, applications, interviews and notifications.	Student sees own data; eligibility and match explanations are displayed from API; valid eligible apply flow works.
F4 — Recruiter	Dashboard, job CRUD/publish/close, skill suggestion review, applicants table, candidate drawer, status updates, interview scheduling and reports.	Approved recruiter can publish; own applicants are paginated/filtered; shortlist and interview workflow works.
F5 — Placement Admin	Overview, recruiter approval, student management, drive monitoring, audit log and analytics.	Admin-only routes work; approving recruiter changes publish permissions; records and reports are correctly scoped.
F6 — Integration hardening	All loading/empty/error states, responsive layouts, accessibility, negative tests, performance and demo data.	Original PS03 acceptance checklist passes on desktop and mobile; no dead controls or fabricated metrics.

12.1 Recommended test accounts and demo data

Use the seeded demo accounts and records from the Build Prompt Pack. The pack specifies one admin, three approved recruiters, one pending recruiter, 25 students across CSE/IT/ECE/MECH, eight jobs with open/draft/closed states, sample resumes, applications across statuses, three interviews, notifications and audit records. Do not hardcode those values into the UI; load them from the backend seed data.

13. PS03 acceptance checklist — frontend evidence

Use this checklist during the final demo. A checkbox means the frontend visibly supports the requirement and the end-to-end behavior has been verified against the backend. A polished screen without working behavior does not pass.

Check	Evidence required	Pass
Role-based access	Student, Recruiter and Placement Admin see separate workspaces; backend blocks wrong-role access.	☐
Student profiles	Student can view/update education, skills, projects, experience and resume.	☐
Job management	Approved recruiter can create, edit, publish and close openings with description, required skills and eligibility.	☐
Eligibility engine	Job detail shows eligible/ineligible and exact branch/CGPA/year/backlog failures from backend.	☐
Job discovery	Student can search/filter by role, location, company and eligibility.	☐
Application workflow	Eligible student applies; application status is visible and progresses through allowed states.	☐
Recruiter dashboard	Recruiter reviews applications, filters candidates, shortlists and updates statuses.	☐
Placement administration	Placement Admin manages student records and monitors active recruitment drives.	☐
Interview details	Candidate and recruiter can see date, time and status information.	☐
PDF extraction	PDF upload status and extracted skills/qualifications appear; invalid files handled cleanly.	☐
Semantic matching	Equivalent skills with different wording can be matched and explained.	☐
Explainable score	Match score, components, matched/missing skills and evidence are visible.	☐
Skill gaps	Missing skills have relevant learning-resource suggestions.	☐
Notifications	Application status and interview schedule notifications appear.	☐
Calendar	Upcoming interviews and recruitment drives have calendar/list representations as supported.	☐
Analytics	In-demand skills, application counts and placement outcomes are shown from API data.	☐
Exports	Candidate lists and recruitment reports can be exported where specified.	☐
Audit logs	Important recruiter/admin actions are inspectable.	☐
Complete workflow brownie point	Recruiter publishes → student applies → recruiter shortlists → status updates.	☐

Check	Evidence required	Pass
Eligibility consistency brownie point	UI shows backend eligibility and cannot bypass server rejection.	☐
Access-control brownie point	Cross-role/cross-owner access is blocked and no restricted data is rendered.	☐
Matching comparison brownie point	Keyword and semantic approaches compared on differently worded sample resumes.	☐
Resume validation brownie point	Missing, malformed and unsupported uploads have clear error handling.	☐
Server-side filtering brownie point	Candidate list pagination, sorting and filtering are verified in API requests.	☐
Realistic demo data brownie point	Seeded students, resumes and jobs make the demonstration reproducible.	☐

14. Copy-ready coding-agent master instruction

How to use Give your coding agent this guide together with the original PS03 PDF and the CampusLink Build Prompt Pack. Keep the original project stack and backend API contract. Ask it to implement one frontend phase at a time and stop for verification after each phase.

You are the senior frontend engineer responsible for implementing CampusLink for PS03 — Nexaura Prompt Battle. Treat the supplied original PS03 and CampusLink Build Prompt Pack as the source specifications, and follow the CampusLink Frontend Instruction Guide for page structure, interactions, design, controls, responsive behavior and acceptance tests.

NON-NEGOTIABLE PRODUCT SCOPE
- The three roles are Student, Recruiter and Placement Admin (the backend may call the last role admin). Do not add Instructor as a role; it is not defined in PS03.
- Implement role-specific dashboards and permissions for these three roles. Frontend route guards are not security boundaries; backend authorization is authoritative.
- Preserve the PS03 workflows: student profile/resume, job postings, eligibility checks, job discovery, resume-to-job matching, applications, shortlisting, interviews, notifications, calendar, analytics, exports and audit logs.
- The UI must explain eligibility failures and match-score evidence. Do not represent AI output as the final eligibility decision or final computed score.

LOCKED CLIENT STACK
- React 18 + Vite + JavaScript (no TypeScript).
- React Router 6, TanStack Query, Axios, Tailwind CSS, react-hook-form, Recharts and react-hot-toast.
- Do not replace or upgrade the stack without explicit permission. Additional 3D dependencies for the landing page must be proposed and approved before installation.

DESIGN AND PAGE REQUIREMENTS
- Build a professional, trustworthy university recruitment product, not a neon/futuristic game interface.
- The landing page must include the requested scroll-driven university journey: as the visitor scrolls, one continuous camera journey moves from a campus overview toward academic/student areas and the placement/recruitment centre. Include explanatory text, clear Sign In/Get Started actions, a skip option, reduced-motion behavior and a static/mobile fallback. Do not make the 3D experience block authentication or the rest of the app.
- Use one unified login page with Student, Recruiter and Placement Admin options. The selected option never grants permissions; use the authenticated server role for redirect and access.
- Build student, recruiter and placement-admin pages and shared components exactly as specified in this guide.
- Every page must have loading, empty and error states as applicable. Every mutation must have pending and success/failure feedback. No dead buttons or fake metrics.

DATA AND SECURITY RULES
- Use Axios with base URL from environment configuration and the agreed Bearer-token mechanism. Use TanStack Query for server state and query invalidation.
- Use server-side pagination, sorting and filtering for lists. Do not fetch all candidates to filter in the browser.
- Display eligibility and match results returned by the backend. Never calculate final eligibility or final match score only in the browser.
- Never call the LLM or embedding model from the client. Use stored matching results and stored summaries returned by the API.
- Students can see only their own profile/applications. Recruiters can see only jobs they own and applicants for those jobs. Placement Admin access follows backend permissions.
- On 401 clear session state and redirect to login; on 403 show access denied without protected data. Treat resume text as untrusted.

IMPLEMENTATION METHOD
1. Inspect the existing repository and current client before editing. Report the actual stack and existing routes; do not overwrite existing work blindly.

2. Implement one frontend phase from this guide at a time. Do not scaffold duplicate pages or unrelated features.
3. Before finishing a phase, run lint/type/build/tests that exist, launch the app, test the relevant routes and report files changed plus exact manual verification steps.
4. Do not move to the next phase until the current phase's exit criteria pass. If a required API endpoint is missing, identify the gap rather than faking the feature with local static data.
5. Keep components reusable, files focused and naming consistent with the existing repository.
6. At the end, run the PS03 acceptance checklist and clearly identify any incomplete item.

Start by inspecting the client and producing a concise implementation plan for F1. Then implement only F1 and stop for verification.

14.1 Final implementation rule

Build the product experience around the original PS03 workflow. The scroll-driven university landing page is the distinctive presentation layer; the decisive product value remains the connected placement workflow with eligibility enforcement, explainable matching, valid applications, recruiter shortlisting, interview scheduling and placement administration. A screen is not complete until it is connected to the relevant API and its negative states have been tested.